SLIIT
Discover Your Future

SE2020 – Web Technologies and Mobile

Lecture 10

NodeJS

SLIIT
FACULTY OF COMPUTING

Agenda

- ❑ Asynchronous JavaScript
- ❑ NodeJS - Introduction
- ❑ Event loop
- ❑ Use Cases
- ❑ Advantages and Disadvantages
- ❑ Package Manager
- ❑ Core Modules & File System

Asynchronous JavaScript

- JS programs run using a single thread. Though there are ways to create new threads, it is considered as **Single threaded language**.
- It is considered asynchronous because it follows a **Non-Blocking I/O (NIO) model**.
- JavaScript is **dynamically typed**.
- JavaScript has an eventing system which manages it's asynchronous operations.
- What is **Non-Blocking I/O (NIO) model**?
  - Non-Blocking I/O means that when JavaScript initiates an operation (like file reading, API requests, and timers), it does not wait for the operation to complete. Instead, it moves on to the next task while the operation executes in the background.

NIO Model - example

```javascript
console.log("Start");

// Non-blocking I/O operation
setTimeout(() => {
    console.log("Data loaded");
}, 2000);

console.log("End");

```

Output:

```
Start
End
undefined
Data loaded
```

Asynchronous Operation

```
function fetchData() {
    setTimeout(() => {
        console.log("Data fetched!"); // Runs after 2 seconds
    }, 2000);
}

function processData() {
    console.log("Processing data...");
}

console.log("Starting fetch operation...");


fetchData();
processData();
```

Output:

Starting fetch operation…
Processing data…

Data fetched!

Is everything look good??

Callback

A **callback** is a function passed as an argument to another function and is executed later, usually after an asynchronous operation completes.

```
function fetchData(callback) {
    setTimeout(() => {
        console.log("Data fetched!"); // Runs after 2 seconds
        callback();
    }, 2000);
}

function processData() {
    console.log("Processing data...");
}

console.log("Starting fetch operation...");

// Call fetchData first
fetchData(processData);

// Running other synchronous code while waiting
for (let i = 0; i < 5; i++) {
    console.log(`Executing synchronous task ${i + 1}`);
}
```

Output:

```
Starting fetch operation...
(Immediately executes the loop)
Executing synchronous task 1
Executing synchronous task 2
Executing synchronous task 3
Executing synchronous task 4
Executing synchronous task 5
(After 2 seconds delay)
Data fetched!
Processing data...
```

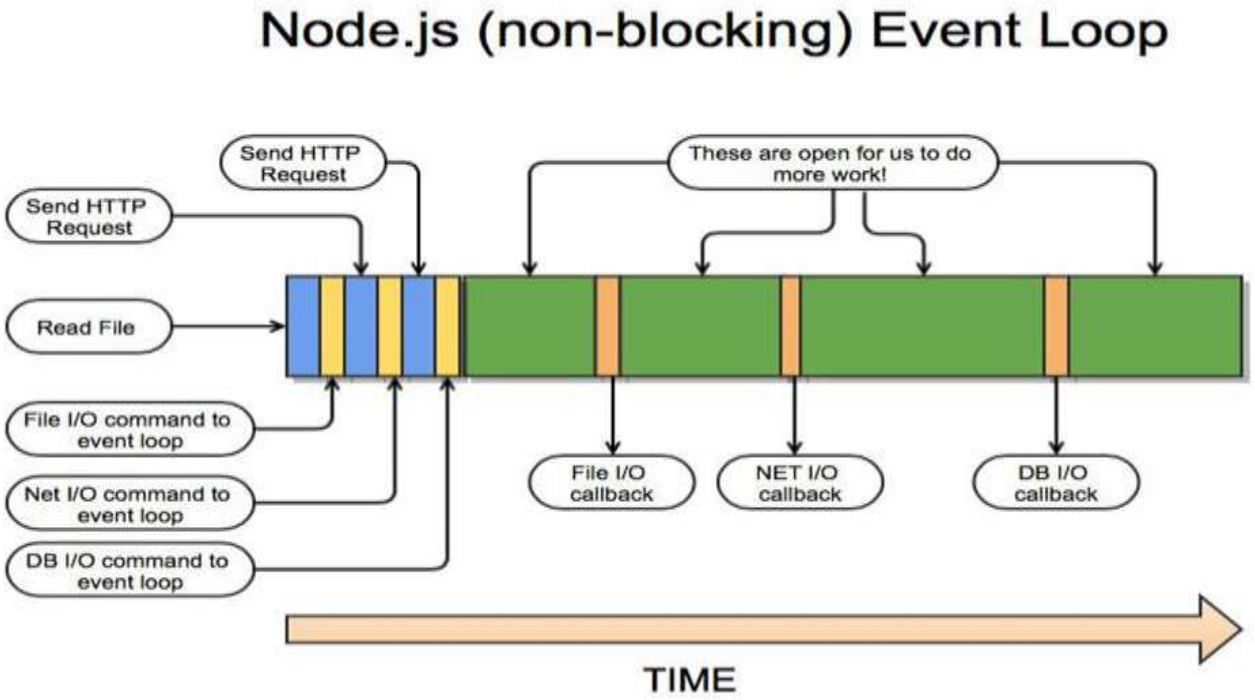
Node JS

- Node Js is a runtime environment that allows JavaScript to be executed on the server side.
- Created by Ryan Dahl to build fast and interactive web applications that need live updates (like chat apps using WebSockets)
- NodeJS is an open source, cross platform runtime environment for server-side and networking applications.
- Uses event-driven, non-blocking I/O model which makes NodeJS lightweight and efficient.
- Ideal for data-intensive real-time applications that run across distributed devices.

Event-Driven Model

- Flow of execution is determined by events, instead of following a sequential execution.
- How works?
  - The system listens for events and, when an event occurs, executes a callback function (event handler) without blocking execution, making event-driven systems more responsive and efficient
- Key components of event driven model:
  - Event emitter
  - Event listener
  - Call back function

Event Loop



The event loop is a core mechanism in Node.js that allows it to handle multiple operations without blocking execution, making it fast and efficient for real-time applications.

Use Cases

- Not the best platform for CPU intensive heavy computational applications.
- Ideal for building fast and scalable network applications.
- NodeJS is capable of handling a huge number of simultaneous connections with high throughput.
- For each connection NodeJS does not spawn new Thread causing max out of memory instead handle all in single thread using non-blocking I/O model.
- NodeJS has achieved over 1 Million concurrent connections.
- Bubbling errors up to NodeJS core event loop will cause crashing the entire program.

Use Cases - Activity

Discuss and write down a real-world success story of companies or projects that have effectively utilized Node.js?

PayPal: 35% Faster Response Time & 2x Requests per Second

Before:

- PayPal's backend was built using Java.
- The development was slow and required separate teams for frontend (JavaScript) and backend (Java).
- Java processes were heavy and slow for handling high-traffic payments.

After Switching to Node.js:

- Unified frontend and backend teams, reducing development time.
- Response time improved by 35%.
- Node.js handled double the number of requests per second compared to Java.
- 33% fewer lines of code, making the system easier to maintain.

Lesson: Node.js improved performance and developer efficiency while handling large-scale transactions.

Use Cases

- I/O bound applications.
- Data streaming applications.
- Data intensive real-time applications.
- JSON APIs based applications.
- Single page applications

Advantages

- Ability to use single programming language from one end of the application to the other end.
- NodeJS applications are easy to scale both horizontally and vertically.
- Delivers improved performance since V8 engine compile the JS code into machine code directly.
- Performance increased via caching modules into memory after the first use.
- Easily extensible and support for common tools like unit testing.
- Well build ‘npm’ package manager and it’s large number of reusable modules.

Disadvantages

- Even though there are number of libraries available, the actual number of robust libraries is comparatively low.
- Not suitable for computationally intensive tasks.
- Asynchronous programming model is complex than synchronous model.

Exercise:

1. What are robust libraries?
2. What are the available robust libraries?
3. Why is the Number of Robust Libraries Low?

Node Package Manager

- Reusable NodeJS components easily available through online repository.
- Build in dependency management, version and scripting mechanism.
- Global installations will be available throughout the system while local installations will only be available for that particular application.
- By default all the dependencies will get installed to ‘node_modules’ directory.
- 'package.json’ contains all information related to the NodeJS application. The file be placed in the root directory of the application.
- ‘package.json’ will contain name, version, author, repository, required node version, scripts and dependencies etc.

Node Package Manager… (cnt)

- To denote the compatible version numbers

Operator	Example	Meaning
<=	"express": "<=4.18.2"	Installs version **4.18.2 or lower**
>=	"express": ">=4.18.2"	Installs version **4.18.2 or higher**
~	"express": "~4.18.2"	Allows **only patch updates** (4.18.x but NOT 4.19.0)
^	"express": "^4.18.2"	Allows **minor & patch updates** (4.x.x but NOT 5.0.0)
*	"express": "*"	Installs **any available version** (not recommended)
1.2.x	"express": "1.2.x"	Installs **any patch version** (1.2.0, 1.2.5, but NOT 1.3.0)
latest	"express": "latest"	Installs the **latest available version** from npm

- There are two types of dependencies,
- Dependencies: needed in runtime (e.g., express, mongoose)
- devDependencies: only for development (e.g., jest, nodemon)

Core Modules

- A module is a reusable block of code. Each file in Node.js is treated as a separate module
- Three Types of Modules:
  - Core/Built-in Modules: Come with Node.js (fs, path, http, os)
  - Local Modules: Files you create yourself
  - Third-party Modules: Installed via npm (npm install chalk)
- Why Modules?
  - Reusability: Use the same code in multiple places
  - Avoid Conflicts: Each module has its own scope
  - Maintainability: Easier to find and fix bugs

Core Modules

- How Modules works

```
const fs = require('fs');          // Core module
const myModule = require('./myModule');  // Local module
const chalk = require('chalk');  // npm module
```

- Create Local Modules

```
JS greetings.js > ...
1   function sayHello(name) {
2       return `Hello, ${name}!`;
3   }
4
5   function sayGoodbye(name) {
6       return `Goodbye, ${name}!`;
7   }
8
9   // Export functions
10  module.exports = {
11      sayHello: sayHello,
12      sayGoodbye: sayGoodbye
13  };
14
```

```
JS index.js > ...
1   const greetings = require('./greetings');
2
3   console.log(greetings.sayHello('Alice'));
4   console.log(greetings.sayGoodbye('Bob'));
5
```

File System (fs Module)

- The fs module is one of the most fundamental and widely-used modules in Node.js. It allows you to work with the file system.
  - Read files, Write files, Delete files, Create directories
- **Data Persistence:** When runs a Node.js program, all data stored in variables exists only in memory (RAM). When the program stops, all data is lost.
- **Configuration Management:** Every application needs settings that can be changed without modifying code.
- **Logging and debugging:** Track What Happens in Your Application.
- **Files uploads and processing:** Handle User-Generated Content.

Example:

1. Create Sample node project and inspect folder structure, package.json file and run sample hello world program.

```js
//Import the chalk package
const chalk = require('chalk');

// Use it!
console.log(chalk.blue('Hello World!'));
console.log(chalk.red.bold('This is red and bold'));
console.log(chalk.green.underline('Success message'));
console.log(chalk.yellow.bgBlue('Warning!'));

// Combining styles
console.log(chalk.blue('Hello') + ' World' + chalk.red('!'));
```

Output:

```
Hello World!
This is red and bold
Success message
Warning!
Hello World!
```

What’s the output ???

```javascript
fs.readFile(__filename, () => {
  console.log("this is readFile 1");
  setImmediate(() => console.log("this is setImmediate 1"));
});

process.nextTick(() => console.log("this is process.nextTick 1"));
Promise.resolve().then(() => console.log("this is Promise.resolve 1"));
setTimeout(() => console.log("this is setTimeout 1"), 0);

for (let i = 0; i < 2000000000; i++) {
  console.log(i);
}
```

SLIIT
Discover Your Future

Thank You